

NEXTPAY

Documentação Técnica e Funcional do Projeto

Aplicação web de carteira digital, transações financeiras e marketplace

Backend Java Spring Boot + Frontend React/Vite

Versão do documento: 1.0

Data: 2026

1. Apresentação do projeto

O NextPay é um sistema web full stack que simula uma plataforma financeira digital integrada a um marketplace interno. A solução permite que usuários se cadastrem, confirmem e-mail, acessem uma conta, consultem saldo, realizem depósitos, saques, transferências, pagamentos via chave PIX, cadastrem produtos, comprem produtos de outros usuários e acompanhem pedidos e histórico de transações.

A documentação foi organizada em formato profissional, semelhante a documentações de APIs comerciais: visão geral, requisitos, recursos disponíveis, endpoints, parâmetros, exemplos de requisição, exemplos de resposta, regras de negócio e instruções de execução.

Item	Descrição
Nome do sistema	NextPay
Tipo de aplicação	Sistema web full stack
Domínio principal	Carteira digital, conta corrente, transações financeiras e marketplace
Backend	Java 21 com Spring Boot
Frontend	React 18 com Vite e TypeScript
Persistência	JPA/Hibernate com H2 em ambiente de desenvolvimento
Comunicação	API REST em JSON
Execução local	Backend na porta 8080 e frontend na porta 5173

2. Escopo e objetivos

2.1 Objetivo geral

Disponibilizar uma aplicação web capaz de centralizar operações financeiras simuladas e compra/venda de produtos entre usuários cadastrados, mantendo histórico das movimentações e regras básicas de segurança e validação.

2.2 Objetivos específicos

- Permitir cadastro de usuários com dados pessoais, endereço e confirmação de e-mail.
- Permitir login e logout de usuários.
- Criar automaticamente uma conta corrente para cada usuário cadastrado.
- Permitir depósito, saque, transferência e PIX por chave de identificação.
- Registrar todas as movimentações financeiras em histórico de transações.
- Permitir cadastro, edição, listagem e exclusão de produtos do marketplace.
- Permitir compra direta e reserva de produtos.
- Atualizar estoque automaticamente após compra, reserva, confirmação ou cancelamento.
- Fornecer um assistente virtual de regras de negócio para apoiar o usuário.
- Disponibilizar interface web responsiva e organizada por páginas funcionais.

3. Tecnologias e dependências

Camada	Tecnologias / bibliotecas	Finalidade
Backend	Java 21, Spring Boot 3.4.0, Maven	Implementação da API REST, regras de negócio e execução do servidor.
Persistência	Spring Data JPA, Hibernate, H2 Database	Mapeamento objeto-relacional e armazenamento dos dados.
Segurança	Spring Security, BCryptPasswordEncoder	Criptografia de senha e configuração de acesso às rotas.
Validação	Jakarta Validation	Validação de dados recebidos nas requisições.
E-mail	Spring Mail	Envio de código de confirmação e notificações.
Integrações	ViaCEP, assistente virtual/RAG	Busca de endereço por CEP e respostas de ajuda ao usuário.
Frontend	React 18, Vite, TypeScript, Axios	Interface do usuário e integração com a API.
UI	Radix UI, Material UI, shadcn/ui, Lucide React	Componentes visuais, ícones e layout.
Relatórios	jsPDF, Recharts	Geração de relatórios e visualizações.
Containerização	Docker, docker-compose, Nginx	Execução do frontend/backend em containers.

4. Arquitetura da solução

A aplicação segue uma organização em camadas. O frontend apresenta as telas e envia requisições HTTP para o backend. O backend recebe as requisições nos controllers, executa regras de negócio nos services, acessa os dados pelos repositories e persiste as entidades no banco.

Usuário

- > Frontend React/Vite
 - > Axios / API REST
 - > Controllers Spring Boot
 - > Services / Regras de negócio
 - > Repositories JPA
 - > Banco H2

Camada	Responsabilidade
Controller	Receber requisições HTTP, extrair parâmetros e retornar respostas.
DTO	Transportar dados específicos de entrada e saída da API.
Service	Concentrar regras de negócio, validações e cálculos.
Repository	Realizar consultas e persistência usando Spring Data JPA.
Model	Representar entidades do domínio e seus relacionamentos.
Config	Centralizar configurações de segurança, IA e RAG.
Frontend	Renderizar telas, controlar estado, chamar endpoints e exibir feedback ao usuário.

5. Estrutura de pastas

5.1 Backend

```
backend/nextpay/projeto_accenture
src/main/java/acc/br/nextpay
  ai/           Assistente de regras de negócio e documentos RAG
  config/       Configurações de segurança, IA e RAG
  controller/   Endpoints REST
  dto/          Objetos de entrada e saída
  model/        Entidades do domínio
  repository/   Interfaces JPA
  service/      Regras de negócio
src/test/java/acc/br/nextpay
  Testes unitários e de controllers/services
Dockerfile
pom.xml
```

5.2 Frontend

```
frontend/frontend-nextpay
src/app
  components/   Componentes reutilizáveis e layout
  contexts/     Contextos de autenticação, banco, marketplace e recompensas
  pages/        Telas principais do sistema
  services/     Configuração do Axios e chamadas à API
  types/        Tipagens TypeScript
  utils/        Funções auxiliares e barramento de eventos
Dockerfile
nginx.conf
package.json
vite.config.ts
```

6. Modelagem de dados

Entidade	Campos principais	Descrição
Usuario	id, nome, cpfCnpj, email, senha, emailConfirmado, tokenConfirmacao, fotoPerfil, pontuacao, totalTransacoes, nivel	Representa o usuário cadastrado e seus dados de acesso.
ContaCorrente	id, numeroConta, saldo, limite, status, usuario	Representa a carteira/conta financeira vinculada ao usuário.
Endereco	id, cep, logradouro, complemento, bairro, localidade, uf, numero, usuario	Armazena o endereço do usuário, preenchido com apoio do ViaCEP.
Produto	id, nome, descricao, categoria, preco, quantidadeEstoque, vendedor, fotoProduto	Representa item cadastrado no marketplace.
Pedido	id, comprador, vendedor, produto, quantidade, valorTotal, status, dataPedido	Representa compra ou reserva feita por um usuário.
Transacao	id, usuario, tipo, valor, data, descricao	Registra movimentações financeiras no histórico.
ItemPedido	id, pedido, produto, quantidade, precoUnitario	Representa item vinculado a pedido, previsto na modelagem.

Enum	Valores identificados	Uso
NivelUsuario	BRONZE, PRATA, OURO, DIAMANTE	Classifica o usuário conforme pontuação/transações.
StatusConta	ATIVO e demais estados previstos no enum	Controla situação da conta corrente.
StatusPedido	RESERVADO, CONFIRMADO, CANCELADO	Controla fluxo do pedido/reserva.
TipoTransacao	DEPOSITO, SAQUE, TRANSFERENCIA, PIX, COMPRA, VENDA	Classifica movimentações financeiras.
TipoUsuario	Tipos previstos no projeto	Representa classificação de usuário quando aplicável.

7. Regras de negócio

7.1 Cadastro e autenticação

- O e-mail do usuário deve ser único no sistema.
- A senha é armazenada utilizando criptografia BCrypt.
- Ao cadastrar o usuário, o sistema gera um token/código de confirmação de e-mail.
- O usuário deve confirmar o e-mail informando o código recebido.
- O login valida e-mail e senha antes de liberar o acesso.

7.2 Carteira digital

- O valor de depósito, saque, PIX ou transferência deve ser maior que zero.
- O saque exige saldo suficiente na conta do usuário.
- A transferência movimenta saldo da conta de origem para a conta de destino.
- O PIX localiza o usuário de destino pela chave informada, que no projeto está associada a dados como e-mail ou CPF/CNPJ.
- Cada movimentação gera registro no histórico de transações.

7.3 Marketplace

- Apenas o dono do produto pode editar ou excluir o produto cadastrado.
- A listagem pública retorna produtos com estoque disponível.
- A compra valida comprador, produto, quantidade, estoque e saldo.
- A reserva reduz temporariamente o estoque do produto.
- O cancelamento da reserva devolve a quantidade reservada ao estoque.
- A confirmação da reserva conclui a compra e registra as transações correspondentes.

7.4 Notificações e gamificação

- O sistema possui serviço de e-mail para confirmação e notificações.
- O usuário possui pontuação, total de transações e nível.
- Mudanças de nível podem acionar notificações ao usuário.

8. Documentação da API REST

Base URL local sugerida: `http://localhost:8080`. Todas as requisições que enviam corpo devem utilizar `Content-Type: application/json`.

8.1 Login

Método	Rota	Descrição
POST	/api/auth/login	Autentica o usuário com e-mail e senha.

Exemplo de requisição

```
{
  "email": "usuario@email.com",
  "senha": "123456"
}
```

Exemplo de resposta

```
{
  "mensagem": "Login realizado com sucesso.",
  "usuarioId": 1,
  "nome": "Neto Souza",
  "email": "usuario@email.com"
}
```

Observações

- Utilizado pelo frontend para iniciar a sessão do usuário.

8.2 Logout

Método	Rota	Descrição
POST	/api/auth/logout	Finaliza a sessão do usuário.

Exemplo de resposta

```
{
  "mensagem": "Logout realizado com sucesso."
}
```

8.3 Cadastrar usuário

Método	Rota	Descrição
POST	/api/usuarios?cep={cep}&numero={numero}	Cadastra um novo usuário e busca endereço pelo CEP.

Exemplo de requisição

```
{
  "nome": "Neto Souza",
  "cpfCnpj": "000000000000",

```

```
{
  "email": "neto@email.com",
  "senha": "123456",
  "fotoPerfil": ""
}
```

Exemplo de resposta

```
{
  "id": 1,
  "nome": "Neto Souza",
  "email": "neto@email.com",
  "emailConfirmado": false
}
```

Observações

- O CEP e o número são enviados como parâmetros de consulta.
- Após o cadastro, o usuário deve confirmar o código enviado por e-mail.

8.4 Confirmar e-mail

Método	Rota	Descrição
POST	/api/usuarios/confirmar-email	Confirma o cadastro a partir do código enviado por e-mail.

Exemplo de requisição

```
{
  "email": "neto@email.com",
  "codigo": "123456"
}
```

Exemplo de resposta

Cadastro realizado com sucesso.

8.5 Listar usuários

Método	Rota	Descrição
GET	/api/usuarios	Retorna todos os usuários cadastrados.

8.6 Buscar usuário por ID

Método	Rota	Descrição
GET	/api/usuarios/{id}	Retorna os dados de um usuário específico.

8.7 Atualizar usuário

Método	Rota	Descrição
PUT	/api/usuarios/{id}	Atualiza os dados do usuário.

Exemplo de requisição

```
{
  "nome": "Neto Souza Atualizado",
  "email": "neto@email.com",
  "fotoPerfil": "base64-ou-url-da-foto"
}
```

8.8 Excluir conta

Método	Rota	Descrição
DELETE	/api/usuarios/{id}?confirmar=true	Exclui a conta do usuário mediante confirmação.

Exemplo de resposta

```
{
  "mensagem": "Conta excluída com sucesso."
}
```

8.9 Consultar saldos

Método	Rota	Descrição
GET	/api/carteiras/saldos	Lista contas correntes e saldos cadastrados.

8.10 Depositar

Método	Rota	Descrição
POST	/api/carteiras/deposito	Realiza depósito na conta do usuário.

Exemplo de requisição

```
{
  "usuarioId": 1,
  "valor": 100.00
}
```

8.11 Sacar

Método	Rota	Descrição
POST	/api/carteiras/saque	Realiza saque da conta do usuário.

Exemplo de requisição

```
{
  "usuarioId": 1,
  "valor": 50.00
}
```

8.12 Transferir

Método	Rota	Descrição
POST	/api/carteiras/transferencia	Transfere valor de um usuário para outro.

Exemplo de requisição

```
{
  "usuarioOrigemId": 1,
  "usuarioDestinoId": 2,
  "valor": 25.00
}
```

Exemplo de resposta

```
{
  "mensagem": "Transferência realizada com sucesso."
}
```

8.13 PIX

Método	Rota	Descrição
POST	/api/carteiras/pix	Realiza transferência por chave PIX.

Exemplo de requisição

```
{
  "usuarioOrigemId": 1,
  "chavePix": "destino@email.com",
  "valor": 30.00
}
```

Exemplo de resposta

```
{
  "mensagem": "PIX realizado com sucesso."
}
```

8.14 Buscar chave PIX

Método	Rota	Descrição
GET	/api/carteiras/pix/buscar?chave={chave}	Busca usuário associado à chave PIX.

Exemplo de resposta

```
{
  "id": "2",
  "nome": "Maria",
  "email": "maria@email.com",
  "cpfCnpj": "11111111111"
}
```

8.15 Cadastrar produto

Método	Rota	Descrição
POST	/api/produtos/usuario/{userId}	Cadastra produto para o usuário vendedor.

Exemplo de requisição

```
{
  "nome": "Mouse Gamer",
  "descricao": "Mouse USB",
  "categoria": "Eletrônicos",
  "preco": 80.00,
  "quantidadeEstoque": 10,
  "fotoProduto": ""
}
```

8.16 Listar produtos disponíveis

Método	Rota	Descrição
GET	/api/produtos	Lista produtos com estoque disponível.

8.17 Listar produtos do usuário

Método	Rota	Descrição
GET	/api/produtos/usuario/{userId}	Lista os produtos cadastrados por um vendedor.

8.18 Editar produto

Método	Rota	Descrição
PUT	/api/produtos/{productId}/usuario/{userId}	Edita produto do usuário vendedor.

Exemplo de requisição

```
{
  "nome": "Mouse Gamer Pro",
  "descricao": "Mouse USB atualizado",
  "categoria": "Eletrônicos",
  "preco": 95.00,
  "quantidadeEstoque": 8
}
```

8.19 Excluir produto

Método	Rota	Descrição
DELETE	/api/produtos/{productId}/usuario/{userId}	Exclui produto se o usuário for o proprietário.

Exemplo de resposta

```
{
  "mensagem": "Produto excluído com sucesso."
}
```

8.20 Comprar produto

Método	Rota	Descrição
POST	/api/pedidos/comprar	Realiza compra direta de produto.

Exemplo de requisição

```
{
  "compradorId": 1,
  "produtoId": 3,
  "quantidade": 2,
  "moedasUsadas": 0
}
```

Exemplo de resposta

```
{
  "id": 10,
  "nomeComprador": "Neto Souza",
  "nomeVendedor": "Maria",
  "nomeProduto": "Mouse Gamer",
  "quantidade": 2,
  "valorTotal": 160.00,
  "status": "CONFIRMADO"
}
```

8.21 Reservar produto

Método	Rota	Descrição
POST	/api/pedidos/reservar	Reserva produto e reduz estoque temporariamente.

Exemplo de requisição

```
{
  "compradorId": 1,
  "produtoId": 3,
  "quantidade": 1
}
```

8.22 Confirmar reserva

Método	Rota	Descrição
PUT	/api/pedidos/{pedidoId}/confirmar	Confirma reserva e finaliza compra.

Exemplo de requisição

```
{
  "compradorId": 1,
  "moedasUsadas": 0
}
```

8.23 Cancelar reserva

Método	Rota	Descrição
PUT	/api/pedidos/{pedidoId}/cancelar	Cancela reserva e devolve o produto ao estoque.

8.24 Listar pedidos

Método	Rota	Descrição
GET	/api/pedidos	Lista todos os pedidos cadastrados.

8.25 Histórico de transações

Método	Rota	Descrição
GET	/api/transacoes/usuario/{usuarioId}	Lista histórico financeiro do usuário.

8.26 Assistente virtual

Método	Rota	Descrição
POST	/api/ajuda/perguntar	Envia uma pergunta para o assistente de regras de negócio.

Exemplo de requisição

```
{
  "pergunta": "Como faço uma transferência?"
}
```

Exemplo de resposta

```
{
  "resposta": "Para realizar uma transferência, informe o usuário de destino e o valor desejado..."
}
```

9. Fluxos principais do sistema

9.1 Fluxo de cadastro e confirmação de e-mail

1. Usuário preenche formulário de cadastro.
2. Frontend envia POST /api/usuarios com CEP e número.
3. Backend consulta ViaCEP e monta endereço.

4. Senha é criptografada com BCrypt.
5. Sistema gera código/token de confirmação.
6. Serviço de e-mail envia o código ao usuário.
7. Usuário informa o código na tela de confirmação.
8. Frontend envia POST /api/usuarios/confirmar-email.
9. Backend valida o código e libera o cadastro.

9.2 Fluxo de transferência

1. Usuário informa origem, destino e valor.
2. Backend valida valor maior que zero.
3. Backend localiza as contas dos usuários.
4. Sistema verifica saldo suficiente na origem.
5. Saldo de origem é debitado.
6. Saldo de destino é creditado.
7. São criadas transações de saída e entrada.
8. Frontend atualiza saldo, extrato e histórico.

9.3 Fluxo de reserva e cancelamento

1. Comprador seleciona produto e quantidade.
2. Sistema valida estoque.
3. Produto é reservado e estoque é reduzido.
4. Pedido fica com status RESERVADO.
5. Se o usuário confirmar, o pagamento é realizado.
6. Se o usuário cancelar, a quantidade retorna ao estoque.

10. Frontend e telas

Página / componente	Função
LandingPage	Tela inicial de apresentação do sistema.
LoginPage	Autenticação do usuário.
CadastroPage	Formulário de cadastro de usuário.
ConfirmarEmailPage	Tela para informar código de confirmação.
DashboardPage	Resumo geral da conta, saldo e informações do usuário.
ContaCorrentePage	Operações de depósito, saque, transferência e PIX.
ExtratoPage	Histórico de movimentações financeiras.
MarketplacePage	Visualização de produtos disponíveis para compra.
ProdutosPage	Gerenciamento dos produtos do usuário.
PedidosPage	Acompanhamento de pedidos do comprador.
PedidosLojaPage	Acompanhamento de pedidos recebidos como vendedor.
PerfilPage	Edição de informações do usuário e foto de perfil.
RelatoriosPage	Visualização/exportação de dados financeiros.
RecompensasPage	Consulta de pontuação, níveis e recompensas.
VirtualAssistant	Componente do assistente virtual integrado à API de ajuda.

Contexto	Responsabilidade
AuthContext	Controla usuário logado, login, logout e persistência da sessão.
BankContext	Gerencia dados de saldo, conta, extrato e operações financeiras.
MarketplaceContext	Controla produtos, compras, reservas e atualização de estoque.
RewardsContext	Gerencia pontuação, recompensas e nível do usuário.

11. Segurança e tratamento de erros

- As senhas são criptografadas com BCrypt antes de serem persistidas.
- A configuração de segurança está centralizada em SecurityConfig.
- O projeto permite CORS para integração entre frontend e backend durante o desenvolvimento.
- O TratadorGlobalDeExcecoes captura RuntimeException e retorna resposta padronizada com campo "erro".
- As principais validações de negócio ficam nos services, evitando duplicação nos controllers.
- Endpoints sensíveis devem ser reforçados futuramente com JWT ou sessão autenticada mais robusta.

Exemplo de erro padronizado:

```
{
  "erro": "Saldo insuficiente."
}
```

12. Integração com e-mail, ViaCEP e assistente virtual

12.1 Serviço de e-mail

O EmailService utiliza JavaMailSender para enviar mensagens. Ele é usado principalmente no fluxo de confirmação de cadastro e pode ser usado para notificações de movimentações, compras, vendas e mudança de nível.

12.2 ViaCEP

O ViaCepService consulta dados de endereço a partir do CEP informado no cadastro do usuário. Com isso, o sistema evita que o usuário precise preencher manualmente todos os campos do endereço.

12.3 Assistente virtual e RAG

O projeto possui classes de configuração para IA e RAG, além do AssistenteRegrasNegocio. O endpoint /api/ajuda/perguntar permite que o frontend envie dúvidas em texto e receba uma resposta de apoio ao usuário.

13. Como executar o projeto

13.1 Pré-requisitos

- Java 21 instalado.
- Maven ou Maven Wrapper disponível.
- Node.js instalado.

- npm instalado.
- Docker opcional para execução containerizada.

13.2 Backend

```
cd backend/nextpay/projeto_accenture
mvn spring-boot:run
```

```
# ou, se estiver no Windows usando o wrapper:
mvnw.cmd spring-boot:run
```

Servidor backend: <http://localhost:8080>

13.3 Frontend

```
cd frontend/frontend-nextpay
npm install
npm run dev
```

Servidor frontend: <http://localhost:5173>

13.4 Docker Compose

```
docker compose up --build
```

A execução por Docker depende das configurações do docker-compose.yml, Dockerfile do backend, Dockerfile do frontend e nginx.conf.

14. Testes sugeridos no Postman/Swagger

Ordem	Teste	Método e rota	Resultado esperado
1	Cadastrar usuário 1	POST /api/usuarios?cep=58400000&numero=100	Usuário criado e e-mail pendente de confirmação.
2	Confirmar e-mail	POST /api/usuarios/confirmar-email	Cadastro confirmado com sucesso.
3	Login	POST /api/auth/login	Retorna usuariold, nome e e-mail.
4	Depositar saldo	POST /api/carteiras/deposito	Saldo do usuário aumenta.
5	Sacar valor válido	POST /api/carteiras/saque	Saldo diminui e transação é registrada.
6	Cadastrar usuário 2	POST /api/usuarios?cep=58400000&numero=200	Segundo usuário criado.
7	Transferir	POST /api/carteiras/transferencia	Origem debita e destino credita.
8	Buscar chave PIX	GET /api/carteiras/pix/buscar?chave=email	Retorna usuário associado.
9	Fazer PIX	POST /api/carteiras/pix	Valor transferido por chave.
10	Cadastrar produto	POST /api/produtos/usuario/{usuariold}	Produto criado com estoque.
11	Listar marketplace	GET /api/produtos	Retorna produtos com estoque.
12	Comprar produto	POST /api/pedidos/comprar	Pedido confirmado, estoque reduzido e transações criadas.
13	Reservar produto	POST /api/pedidos/reservar	Pedido reservado e estoque reduzido.
14	Cancelar reserva	PUT /api/pedidos/{id}/cancelar	Estoque devolvido.

15	Consultar extrato	GET /api/transacoes/usuario/{usuarioId}	Histórico retorna movimentações.
----	-------------------	---	----------------------------------

15. Melhorias futuras

- Implementar autenticação JWT para proteger melhor os endpoints.
- Adicionar documentação Swagger/OpenAPI gerada automaticamente.
- Implementar logs estruturados e auditoria de transações.
- Adicionar recuperação de senha por e-mail.
- Adicionar painel administrativo com métricas de usuários, produtos, pedidos e transações.
- Padronizar respostas da API com objeto contendo sucesso, mensagem, dados e erros.

16. Considerações finais

O NextPay apresenta uma proposta completa de sistema financeiro simulado com integração a marketplace. O projeto demonstra domínio de conceitos importantes de desenvolvimento de software, como arquitetura em camadas, API REST, persistência com JPA, autenticação, criptografia de senhas, integração com serviços externos, componentização no frontend e tratamento de regras de negócio.

A documentação apresentada pode ser usada como base para entrega acadêmica, apresentação do projeto, explicação técnica ao grupo, criação de README profissional ou preparação de testes no Postman. Com ajustes futuros de autenticação, banco de produção, documentação Swagger e maior cobertura de testes, o sistema pode evoluir para uma solução ainda mais robusta e próxima de um ambiente real.